

PenguWave — Threat Thinking

Written before any backend code. The point is to figure out what can go wrong first, so every later decision has a reason.

What we're protecting

PenguWave is a portal where analysts view security events and admins manage user accounts. Three things matter:

1. **The event data** — internal hostnames, IPs, and incident details. Useful to an attacker mapping the environment.
2. **User accounts** — especially admin accounts, which can create users and change roles.
3. **Whatever proves the user is logged in** — a token or cookie. Stealing it means impersonating that user.

Who might attack it

- **Someone with no account**, trying to guess passwords or hit the API without logging in.
- **A logged-in analyst or viewer**, trying to call admin-only endpoints, or sneak **role: "admin"** into a request that's supposed to update something else.
- **Anyone who can run JavaScript in the page** (via XSS) — they can steal a token from **localStorage**, or ride the user's session if we use cookies.

The starter frontend already has weak spots: it logs credentials to the console, keeps tokens in **localStorage**, renders event text as HTML, and shows the Users page without checking the role. The real defences has to live on the **backend**.

What I plan to protect against

- **Password guessing** — hash passwords with argon2, rate-limit the login endpoint, and return the same error message whether the email exists or not.
- **Stolen sessions** — keep the session id in an httpOnly, Secure, SameSite cookie so JavaScript can't read it; expire it quickly; delete it on logout.
- **Privilege escalation** — check roles on the **server** for every admin route. Reject unknown fields on update requests so nobody can smuggle in **role: "admin"**.
- **Reading data they shouldn't** — authorize every protected resource. Never return password hashes.
- **XSS** — render all user-controlled text as plain text, not HTML.
- **CSRF** — **SameSite** on the cookie blocks cross-site requests in modern browsers.
- **Leaks in logs and errors** — consistent error shape, no passwords in logs (the starter currently logs them; that gets removed).